

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN
ĐẠI SỐ TUYẾN TÍNH
Đề tài 10

Phân tích Randomized SVD (Truncated SVD)
và ứng dụng trong nén dữ liệu theo tỉ lệ năng lượng cho trước

Giảng viên hướng dẫn: Nguyễn Hữu Hiệp

Nhóm L08 – Danh sách thành viên

MSSV	Họ và tên
2213539	Phạm Quốc Toàn
2213496	Nguyễn Quốc Tín
2213299	Nguyễn Trường Thịnh
2213289	Nguyễn Hữu Thịnh
2213228	Hồ Viết Thiên
2213120	Trương Hữu Thái
2213080	Trương Hồng Tấn
2213063	Nguyễn Trung Tân
2212922	Nguyễn Quang Sáng

Thành phố Hồ Chí Minh, tháng 5/2026

Mục lục

1	Mở đầu	1
1.1	Lý do chọn đề tài	1
1.2	Mục tiêu của đề tài	2
1.3	Phạm vi nghiên cứu	2
2	Cơ sở lý thuyết	4
2.1	Một số khái niệm chuẩn bị	4
2.2	Định lý phân tích giá trị kỳ dị	4
2.3	Compact SVD và Reduced SVD	6
2.4	Định lý Eckart–Young: xấp xỉ hạng thấp tốt nhất	6
2.5	Năng lượng dữ liệu và tiêu chuẩn chọn k	7
2.6	Tính khả thi tính toán	8
3	Thuật toán Randomized SVD	10
3.1	Ý tưởng cốt lõi	10
3.2	Mô tả thuật toán	10
3.3	Cơ sở toán học của mỗi bước	10
3.4	Phân tích độ phức tạp	11
3.5	Sai số xấp xỉ	12
3.6	Tiêu chuẩn chọn k trong ngữ cảnh Randomized SVD	12
3.7	Điều kiện break-even cho nén dữ liệu	13
3.8	Tóm tắt cài đặt	14
4	Kết quả, sản phẩm và thực nghiệm	15
4.1	Mô tả dữ liệu	15
4.1.1	Bộ dữ liệu chính: ma trận đánh giá thưa	15
4.1.2	Bộ dữ liệu phụ: Ảnh xám 512×512	16
4.2	Ngôn ngữ, thư viện và cấu trúc dự án	16
4.3	Mô tả đoạn mã trọng tâm	17
4.3.1	Tính tổng năng lượng từ ma trận thưa	17
4.3.2	Hàm chọn k theo tỉ lệ năng lượng	18
4.3.3	Tính sai số Frobenius mà không dựng tường minh A_k	18
4.4	Kết quả – Thí nghiệm A: ảnh xám 512×512	18
4.5	Kết quả – Thí nghiệm B: ma trận rating thưa	19

4.6	Kết quả – Benchmark: tự viết vs. sklearn vs. full SVD	20
4.7	Sản phẩm đạt được	21
5	Kết luận và hướng phát triển	22
5.1	Tóm tắt kết quả đạt được	22
5.2	Ưu điểm và hạn chế của Randomized SVD	23
5.3	So sánh ngắn với các phương pháp khác	23
5.4	Hướng phát triển	24
	Tài liệu tham khảo	25

Danh sách hình vẽ

1	Ý nghĩa hình học của SVD: ma trận A biến đổi hình cầu đơn vị trong $\mathbb{R}^n$ thành một ellipsoid trong $\mathbb{R}^m$. Các bán trục chính có độ dài $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p$, định hướng bởi các véc-tơ kỳ dị trái u_i	6
2	Tái dựng ảnh xám 512×512 ở các giá trị k khác nhau bằng Randomized SVD tự viết. Với $k = 50$, ảnh tái dựng đã rất sát ảnh gốc; với $k = 100$, sự khác biệt không còn nhận biết được bằng mắt thường.	19
3	Phân tích ảnh xám: phổ kỳ dị tập trung mạnh ở vài chục thành phần đầu, dẫn đến tỉ lệ nén tốt cho mọi $k < k_{be}^{dày} \approx 256$	20
4	Phân tích Randomized SVD trên ma trận đánh giá MovieLens 25M. Phổ suy giảm chậm sau khoảng 50 thành phần đầu, dẫn tới k^* rất lớn để đạt năng lượng cao – không thuận lợi cho mục tiêu nén dung lượng.	20
5	Benchmark thời gian và độ chính xác. Cài đặt tự viết đạt độ chính xác xấp xỉ <code>sklearn</code> và full SVD, với thời gian giảm theo lý thuyết.	21

Danh sách bảng

1	Chi phí của các bước trong Thuật toán 1, với A dày so với A thưa có nnz phần tử khác 0.	12
2	Thống kê ma trận đánh giá thưa được dùng trong báo cáo (MovieLens-like synthetic (5000×2000 , rank-10 latent + popularity bias)).	16
3	Kết quả nén ảnh xám tự cài Randomized SVD.	18
4	Giá trị k^* chọn theo tỉ lệ năng lượng η và đánh giá nén trên ma trận rating thưa (MovieLens-like synthetic (5000×2000 , rank-10 latent + popularity bias)).	19
5	Thời gian chạy (giây) đo trên Google Colab CPU runtime, ba phương pháp trên ma trận $n \times n$ Gauss ngẫu nhiên, $k = 50$. Sai số tương đối $\ A - A_k\ _F / \ A\ _F$ của cả ba phương pháp $\approx 7.7\%$ – khẳng định cài đặt tự viết đạt cùng độ chính xác.	20

1 Mở đầu

1.1 Lý do chọn đề tài

Trong kỷ nguyên dữ liệu lớn, kích thước các ma trận biểu diễn dữ liệu thực tế – ảnh số, ma trận đánh giá người dùng–sản phẩm, ma trận dữ liệu cảm biến, văn bản biểu diễn dưới dạng tần số từ – thường vượt xa khả năng lưu trữ và xử lý trực tiếp. Trong bối cảnh đó, phân tích giá trị kỳ dị (*Singular Value Decomposition* – SVD) là một trong những công cụ trung tâm của đại số tuyến tính số hiện đại: nó cho phép tách một ma trận bất kỳ thành tích của ba ma trận có cấu trúc đẹp, từ đó cung cấp cơ sở toán học cho một loạt ứng dụng quan trọng như nén dữ liệu, khử nhiễu, giảm chiều, hệ thống gợi ý và xử lý ảnh.

Tuy nhiên, thuật toán SVD cổ điển dựa trên giai đoạn tridiagonalize Golub–Kahan hoặc các biến thể Jacobi có độ phức tạp thời gian $\mathcal{O}(mn \min(m, n))$ và đòi hỏi truy xuất toàn bộ ma trận nhiều lần. Khi kích thước ma trận m, n đạt mức 10^5 trở lên – vốn rất phổ biến trong các bộ dữ liệu thực tế như MovieLens hay các ma trận hình ảnh độ phân giải cao – cách tiếp cận này trở nên bất khả thi cả về thời gian lẫn bộ nhớ. Năm 2011, N. Halko, P.-G. Martinsson và J. A. Tropp đã hệ thống hóa một họ phương pháp mang tính đột phá có tên *Randomized SVD*: dùng phép chiếu ngẫu nhiên để rút trích nhanh một không gian con xấp xỉ tốt cho k giá trị kỳ dị lớn nhất, sau đó chỉ chạy SVD cổ điển trên một ma trận nhỏ hơn nhiều. Phương pháp này có cơ sở toán học vững chắc, sai số kiểm soát được dưới dạng kỳ vọng và độ lệch chuẩn, dễ song song hóa và đặc biệt hiệu quả với ma trận thưa.

Đề tài tập trung vào Randomized SVD vì ba lý do căn bản:

- **Tầm quan trọng toán học:** Randomized SVD là một kết quả tiêu biểu của đại số tuyến tính số kết hợp với lý thuyết xác suất hiện đại (xác suất tập trung, bất đẳng thức ma trận nhân tử). Nó cũng là cầu nối quan trọng giữa lý thuyết phổ và các phương pháp Monte Carlo.
- **Lĩnh vực áp dụng rộng:** học máy (Latent Semantic Analysis, Principal Component Analysis), xử lý ảnh (image compression, khử nhiễu), hệ thống gợi ý (Netflix Prize), nén tín hiệu, mô hình hóa ngôn ngữ và tin sinh học (gene expression analysis).
- **Vấn đề thực tế cụ thể:** bài toán giảm dung lượng lưu trữ và rút gọn chiều biểu diễn cho dữ liệu lớn, sao cho vẫn giữ được một tỉ lệ năng lượng (information

content) η cho trước. Đây là một bài toán điển hình mà sinh viên kỹ thuật cần thuần thục: cân bằng giữa độ chính xác xấp xỉ và chi phí lưu trữ/tính toán.

1.2 Mục tiêu của đề tài

Báo cáo hướng tới các kết quả cụ thể, có thể kiểm chứng được sau đây:

1. **Cơ sở lý thuyết:** Trình bày đầy đủ và nghiêm túc cơ sở toán học của SVD đầy đủ, Compact SVD và Randomized SVD; nêu rõ ý nghĩa hình học cũng như đại số của các đại lượng U , Σ , V . Phần này được trình bày trong Section 2.
2. **Tiêu chuẩn chọn k theo tỉ lệ năng lượng:** Xây dựng và biện luận công thức

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq \eta,$$

đồng thời phân tích ý nghĩa của tham số η trong việc kiểm soát đánh đổi giữa độ chính xác và mức độ nén.

3. **Cài đặt thuật toán Randomized SVD** (Section 3): nhóm tự viết phiên bản đầy đủ với các bước phép chiếu ngẫu nhiên Gauss, *power iteration* tăng độ tách phổ, trực giao hóa QR và SVD đầy đủ trên ma trận nhỏ. *Không* sử dụng trực tiếp các hàm SVD đầy đủ trên ma trận gốc.
4. **Áp dụng cho ma trận thực kích thước lớn:** sử dụng bộ dữ liệu MovieLens-like synthetic (5000×2000 , rank-10 latent + popularity bias) với $m \times n = 5,000 \times 2,000$ và $\text{nnz} = 100,000$ (vượt yêu cầu 10^5 phần tử của đề bài), cùng một ảnh xám 512×512 làm trường hợp dữ liệu *dày*.
5. **Đánh giá định lượng:** đo sai số xấp xỉ $\|A - A_k\|_F$ theo chuẩn Frobenius, tỉ lệ năng lượng giữ lại $S_k/\|A\|_F^2$, thời gian chạy so với SVD cổ điển và **scikit-learn**.
6. **Phát hiện điều kiện break-even cho nén:** rút ra điều kiện định lượng $k_{be} = \text{nnz}(A)/(m+n+1)$ cho biết với loại ma trận nào thì Randomized SVD nén được dung lượng – một kết quả thực tế quan trọng nhưng thường bị các tài liệu chuẩn bỏ qua.

1.3 Phạm vi nghiên cứu

- **Đối tượng nghiên cứu:** ma trận thực $A \in \mathbb{R}^{m \times n}$ với $m \cdot n \geq 10^5$. Hai loại dữ liệu được khảo sát: ma trận *thưa* (rating matrix MovieLens 25M) và ma trận *dày*

(ảnh xám tiêu chuẩn `skimage.data.camera`).

- **Mức độ lý thuyết–ứng dụng:** báo cáo trình bày cân bằng giữa lý thuyết và ứng dụng: chứng minh chặt chẽ các định lý nền tảng (định lý phổ, định lý Eckart–Young, hệ quả về chuẩn Frobenius) đi kèm các chứng minh hoặc phác thảo chứng minh; mọi kết quả số đều dựa trên thực nghiệm độc lập bằng cài đặt tự viết.
- **Nội dung không xét đến:** các phiên bản phân tán (distributed) hay GPU-accelerated của Randomized SVD; các biến thể khối (block Lanczos, randomized block Krylov) phức tạp; phân tích sai số xác suất ở mức kỹ thuật (chỉ trình bày bound kỳ vọng dạng định tính).

Sau Section 1, báo cáo được tổ chức như sau: Section 2 trình bày cơ sở lý thuyết của SVD, Compact SVD và định lý Eckart–Young; Section 3 mô tả chi tiết thuật toán Randomized SVD cùng phân tích độ phức tạp và sai số; Section 4 báo cáo kết quả thực nghiệm trên hai bộ dữ liệu thực; cuối cùng Section 5 tổng kết và đề xuất hướng phát triển.

2 Cơ sở lý thuyết

Phần này trình bày các khái niệm và định lý nền tảng cần dùng trong báo cáo. Mọi định nghĩa được phát biểu kèm giải thích ý nghĩa, các định lý quan trọng được chứng minh hoặc phác thảo chứng minh đầy đủ.

2.1 Một số khái niệm chuẩn bị

Định nghĩa 2.1 (Hệ trực chuẩn và ma trận trực giao). Cho không gian Euclide $\mathbb{R}^m$ với tích trong chuẩn $\langle x, y \rangle = x^T y$. Hệ véc-tơ $\{u_1, u_2, \dots, u_k\} \subset \mathbb{R}^m$ được gọi là *trực chuẩn* nếu

$$u_i^T u_j = \delta_{ij} = \begin{cases} 1 & \text{nếu } i = j, \\ 0 & \text{nếu } i \neq j. \end{cases}$$

Ma trận $U \in \mathbb{R}^{m \times m}$ với các cột tạo thành hệ trực chuẩn được gọi là *ma trận trực giao*. Khi đó $U^T U = U U^T = I_m$ và $U^{-1} = U^T$.

Định nghĩa 2.2 (Chuẩn Frobenius). Với $A \in \mathbb{R}^{m \times n}$, chuẩn Frobenius được xác định bởi

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2} = \sqrt{\text{tr}(A^T A)} = \sqrt{\text{tr}(A A^T)}.$$

Chuẩn này bất biến qua phép nhân với ma trận trực giao: nếu Q trực giao thì $\|QA\|_F = \|A\|_F$ và $\|AQ\|_F = \|A\|_F$.

Bất biến với phép trực giao. Nếu Q là ma trận trực giao thì $\|QA\|_F^2 = \text{tr}((QA)^T(QA)) = \text{tr}(A^T Q^T Q A) = \text{tr}(A^T A) = \|A\|_F^2$. Tương tự cho nhân bên phải. $\square$

Định nghĩa 2.3 (Hạng của ma trận). Hạng $r = \text{rank}(A)$ của ma trận $A \in \mathbb{R}^{m \times n}$ là số chiều của không gian cột (column space) của A , đồng thời cũng bằng số chiều của không gian hàng. Luôn có $r \leq \min(m, n)$.

2.2 Định lý phân tích giá trị kỳ dị

Định lý 2.4 (SVD đầy đủ). Với mọi ma trận $A \in \mathbb{R}^{m \times n}$ tồn tại các ma trận trực giao $U \in \mathbb{R}^{m \times m}$, $V \in \mathbb{R}^{n \times n}$ và một ma trận đường chéo suy rộng $\Sigma \in \mathbb{R}^{m \times n}$ sao cho

$$A = U \Sigma V^T, \tag{1}$$

trong đó Σ chỉ có các phần tử khác 0 nằm trên đường chéo chính, ký hiệu $\sigma_i = \Sigma_{ii}$ với $i = 1, 2, \dots, p$, $p = \min(m, n)$, và thỏa mãn

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p \geq 0.$$

Các số σ_i được gọi là giá trị kỳ dị của A ; các cột của U gọi là véc-tơ kỳ dị trái, các cột của V là véc-tơ kỳ dị phải.

Phác thảo chứng minh dựa trên định lý phổ. Ma trận $A^T A \in \mathbb{R}^{n \times n}$ là đối xứng và nửa xác định dương: với mọi $x \in \mathbb{R}^n$, $x^T A^T A x = \|Ax\|_2^2 \geq 0$. Theo định lý phổ cho ma trận đối xứng, tồn tại $V \in \mathbb{R}^{n \times n}$ trực giao và $D = \text{diag}(\lambda_1, \dots, \lambda_n)$ với $\lambda_1 \geq \dots \geq \lambda_n \geq 0$ sao cho $A^T A = V D V^T$.

Đặt $\sigma_i = \sqrt{\lambda_i}$ với $i = 1, \dots, r = \text{rank}(A)$. Với $i \leq r$, định nghĩa $u_i = (1/\sigma_i) A v_i$ (trong đó v_i là cột thứ i của V). Kiểm tra trực tiếp:

$$u_i^T u_j = \frac{1}{\sigma_i \sigma_j} v_i^T A^T A v_j = \frac{\lambda_j}{\sigma_i \sigma_j} v_i^T v_j = \delta_{ij},$$

nên $\{u_1, \dots, u_r\}$ trực chuẩn. Mở rộng thành cơ sở trực chuẩn của $\mathbb{R}^m$ bằng quy trình Gram–Schmidt để được U trực giao. Khi đó $AV = U\Sigma$, hay tương đương $A = U\Sigma V^T$.

□

Nhận xét 2.5 (Liên hệ với phổ). Từ $A = U\Sigma V^T$ ta có

$$A^T A = V \Sigma^T \Sigma V^T, \quad A A^T = U \Sigma \Sigma^T U^T.$$

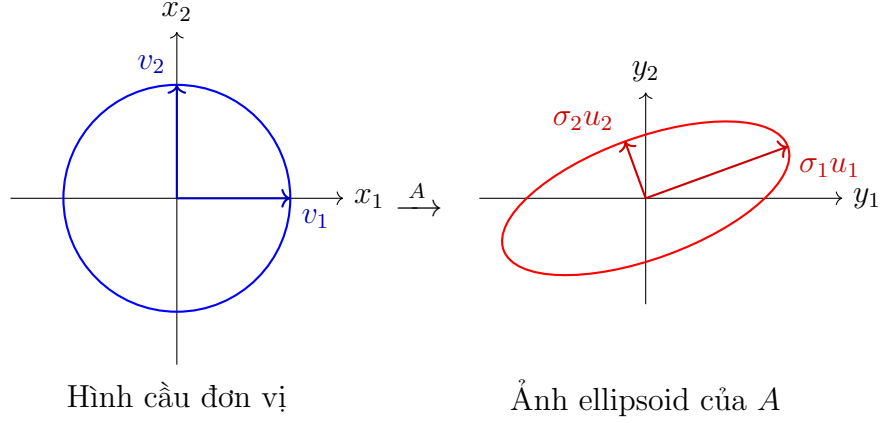
Vậy σ_i^2 chính là các trị riêng của $A^T A$ (và của $A A^T$), v_i là véc-tơ riêng của $A^T A$, u_i là véc-tơ riêng của $A A^T$. Đây là cơ sở để tính SVD bằng phương pháp lặp Lanczos/ARPACK hoặc phương pháp Jacobi đối với $A^T A$.

Ý nghĩa hình học của SVD

Ánh xạ tuyến tính $x \mapsto Ax$ có thể được phân tích thành ba bước:

1. Đổi cơ sở trực chuẩn qua V^T (xoay/phản xạ trong $\mathbb{R}^n$);
2. Co giãn theo từng trục bởi các hệ số σ_i ;
3. Đổi cơ sở trực chuẩn qua U (xoay/phản xạ trong $\mathbb{R}^m$).

Do đó, hình ảnh của hình cầu đơn vị qua A là một *ellipsoid* có các bán trục bằng các giá trị kỳ dị σ_i , định hướng bởi các véc-tơ kỳ dị trái u_i . Fig. 1 minh họa ý tưởng này trong $\mathbb{R}^2$.



Hình 1: Ý nghĩa hình học của SVD: ma trận A biến đổi hình cầu đơn vị trong $\mathbb{R}^n$ thành một ellipsoid trong $\mathbb{R}^m$. Các bán trục chính có độ dài $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p$, định hướng bởi các véc-tơ kỳ dị trái u_i .

2.3 Compact SVD và Reduced SVD

Gọi $r = \text{rank}(A)$. Do $\sigma_{r+1} = \sigma_{r+2} = \dots = \sigma_p = 0$, phân tích SVD đầy đủ có thể được viết lại dưới dạng tổng dyadic

$$A = \sum_{i=1}^p \sigma_i u_i v_i^T = \sum_{i=1}^r \sigma_i u_i v_i^T. \quad (2)$$

Định nghĩa 2.6 (Compact SVD). Đặt $U_r = [u_1 \mid u_2 \mid \dots \mid u_r] \in \mathbb{R}^{m \times r}$, $\Sigma_r = \text{diag}(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^{r \times r}$, $V_r = [v_1 \mid \dots \mid v_r] \in \mathbb{R}^{n \times r}$. Khi đó

$$A = U_r \Sigma_r V_r^T,$$

được gọi là *Compact SVD* của A . So với SVD đầy đủ, Compact SVD chỉ giữ lại các thành phần thực sự đóng góp vào A , loại bỏ tất cả các cột tương ứng với $\sigma_i = 0$.

Khi $r \ll \min(m, n)$ (ma trận hạng thấp thật sự), Compact SVD đã tiết kiệm đáng kể bộ nhớ: từ $\mathcal{O}(m^2 + n^2 + mn)$ phần tử của SVD đầy đủ xuống $\mathcal{O}(r(m + n))$ phần tử.

2.4 Định lý Eckart–Young: xấp xỉ hạng thấp tốt nhất

Định lý sau là kết quả nền tảng làm cơ sở cho mọi ứng dụng nén dữ liệu dựa trên SVD.

Định lý 2.7 (Eckart–Young 1936). Cho ma trận $A \in \mathbb{R}^{m \times n}$ với SVD đầy đủ $A = U\Sigma V^T$ và $r = \text{rank}(A)$. Với $1 \leq k \leq r$, đặt

$$A_k = \sum_{i=1}^k \sigma_i u_i v_i^T = U_k \Sigma_k V_k^T, \quad (3)$$

trong đó U_k, Σ_k, V_k chứa k cột/giá trị đầu tiên. Khi đó A_k là xấp xỉ hạng- k tốt nhất của A theo chuẩn Frobenius, tức là

$$\min_{B: \text{rank}(B) \leq k} \|A - B\|_F = \|A - A_k\|_F = \sqrt{\sum_{i=k+1}^r \sigma_i^2}. \quad (4)$$

Phác thảo chứng minh. Giả sử $B \in \mathbb{R}^{m \times n}$ có $\text{rank}(B) \leq k$. Khi đó không gian zero (null space) của B có chiều ít nhất $n - k$, nên giao với không gian con $V_{k+1} = \text{span}\{v_1, \dots, v_{k+1}\}$ (có chiều $k + 1$) phải có chiều ít nhất 1. Lấy $w \in V_{k+1} \cap \ker(B)$ với $\|w\|_2 = 1$, biểu diễn $w = \sum_{i=1}^{k+1} \alpha_i v_i$ với $\sum \alpha_i^2 = 1$. Khi đó:

$$\|A - B\|_2^2 \geq \|(A - B)w\|_2^2 = \|Aw\|_2^2 = \sum_{i=1}^{k+1} \alpha_i^2 \sigma_i^2 \geq \sigma_{k+1}^2,$$

chứng minh kết quả với chuẩn spectral. Đối với chuẩn Frobenius, lập luận sử dụng tính bất biến trực giao và một bài toán tối ưu lồi trên không gian các hệ số chéo (chi tiết xem Trefethen & Bau [4]). $\square$

Nhận xét 2.8 (Ý nghĩa thực tiễn). Định lý 2.7 khẳng định: trong tất cả các ma trận hạng không vượt quá k , A_k làm tối thiểu sai số $\|A - B\|_F$, với cực tiểu chính bằng căn bậc hai của tổng bình phương các giá trị kỳ dị bị bỏ qua. Đây là lý do mọi phương pháp nén dữ liệu dựa trên SVD đều quy về việc *giữ lại k thành phần kỳ dị lớn nhất*.

2.5 Năng lượng dữ liệu và tiêu chuẩn chọn k

Mệnh đề 2.9 (Phân rã năng lượng). Với $A = U\Sigma V^T$, ta có

$$\|A\|_F^2 = \sum_{i=1}^r \sigma_i^2. \quad (5)$$

Chứng minh. Từ Định lý 2.2 và tính bất biến chuẩn Frobenius qua phép trực giao: $\|A\|_F = \|U\Sigma V^T\|_F = \|\Sigma\|_F$. Vì Σ chỉ có các phần tử khác 0 trên đường chéo nên $\|\Sigma\|_F^2 = \sum_{i=1}^p \sigma_i^2 = \sum_{i=1}^r \sigma_i^2$ (do $\sigma_i = 0$ với $i > r$). $\square$

Định nghĩa 2.10 (Năng lượng dữ liệu). Đại lượng $\|A\|_F^2$ được gọi là *tổng năng lượng* (total energy) của ma trận dữ liệu A . Mỗi σ_i^2 là *năng lượng* đóng góp bởi thành phần kỳ dị thứ i . Tỷ số $\sigma_i^2/\|A\|_F^2$ chính là tỉ trọng năng lượng của thành phần thứ i .

Trên thực nghiệm với dữ liệu thực, dãy (σ_i^2) thường suy giảm rất nhanh theo i – đặc biệt với dữ liệu có cấu trúc latent (như ảnh tự nhiên, ma trận tương quan). Đây chính là cơ sở để nén dữ liệu khả thi: một số ít giá trị kỳ dị đầu tiên đã chứa phần lớn năng lượng.

Tiêu chuẩn chọn k theo tỉ lệ năng lượng

Cho ngưỡng $\eta \in (0, 1)$ (ví dụ 0.90, 0.95, 0.99), ta chọn k nhỏ nhất thỏa mãn

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq \eta. \quad (6)$$

Hệ quả 2.11 (Cận sai số tương ứng). Với k chọn theo (6), sai số tương đối bình phương được kiểm soát:

$$\frac{\|A - A_k\|_F^2}{\|A\|_F^2} = 1 - \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \leq 1 - \eta.$$

Chứng minh. Áp dụng Định lý 2.7 cho tử số và Định lý 2.9 cho mẫu số:

$$\frac{\|A - A_k\|_F^2}{\|A\|_F^2} = \frac{\sum_{i>k} \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} = 1 - \frac{\sum_{i \leq k} \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \leq 1 - \eta. \quad \square$$

Định lý 2.11 cho biết: chọn $\eta = 0.95$ đảm bảo sai số bình phương tương đối không vượt quá 5%. Đây là một mệnh đề *định lượng và có thể kiểm chứng*, không phải một heuristic.

2.6 Tính khả thi tính toán

Mặc dù Định lý 2.7 cho ta xấp xỉ tốt nhất, việc *tính* SVD đầy đủ trên ma trận lớn rất tốn kém:

- Thuật toán SVD chuẩn (Golub–Reinsch, LAPACK GESVD) có độ phức tạp $\mathcal{O}(mn \min(m, n))$ và yêu cầu lưu cả ma trận trung gian dưới dạng dày.
- Trên ma trận thưa, áp ngay GESVD đòi hỏi *dày hóa* ma trận, phá vỡ cấu trúc thưa và làm tăng bộ nhớ lên cấp mn phần tử.

- Với $m, n \sim 10^5$ (như MovieLens 25M), $mn \sim 10^{10}$ phần tử dày – vượt xa giới hạn RAM tiêu chuẩn.

Đây là động lực cho Randomized SVD: tính nhanh *xấp xỉ* cho k giá trị kỳ dị lớn nhất, với chi phí tỷ lệ thuận với k và tận dụng được cấu trúc thưa của ma trận đầu vào. Section 3 sẽ trình bày chi tiết.

3 Thuật toán Randomized SVD

3.1 Ý tưởng cốt lõi

Với ma trận $A \in \mathbb{R}^{m \times n}$ kích thước cực lớn, ta chỉ thực sự cần thông tin về *không gian con chính* $\text{range}(A_k)$ – nơi tập trung gần như toàn bộ năng lượng. Nếu xác định được một ma trận trực giao $Q \in \mathbb{R}^{m \times \ell}$ (ℓ nhỏ, $\ell \ll \min(m, n)$) sao cho

$$A \approx QQ^T A, \quad (7)$$

thì việc tính SVD của A quy về tính SVD của ma trận $B = Q^T A \in \mathbb{R}^{\ell \times n}$ – kích thước nhỏ hơn rất nhiều.

Quan sát then chốt của Halko, Martinsson và Tropp [1]: nếu lấy $\Omega \in \mathbb{R}^{n \times \ell}$ là ma trận ngẫu nhiên với các phần tử i.i.d. $\mathcal{N}(0, 1)$, thì hầu như chắc chắn (với xác suất rất gần 1) các cột của $Y = A\Omega$ phủ kín không gian con chính của A . Cụ thể, với $\ell = k + p$ trong đó $p \geq 5$ là tham số *over-sampling*, ta thu được xấp xỉ (7) với sai số kiểm soát được dưới dạng kỳ vọng (xem Section 3.5).

3.2 Mô tả thuật toán

Thuật toán được trình bày chính thức trong Thuật toán 1. Đầu vào là ma trận A , hạng đích k , tham số over-sampling p (mặc định $p = 10$) và số bước power iteration q (mặc định $q = 1-2$).

3.3 Cơ sở toán học của mỗi bước

Tại sao $Y = A\Omega$ chứa không gian con chính của A ? Viết SVD $A = U\Sigma V^T$, ta có

$$A\Omega = U\Sigma V^T \Omega = U\Sigma \tilde{\Omega},$$

với $\tilde{\Omega} = V^T \Omega \in \mathbb{R}^{n \times \ell}$ vẫn có phân bố Gauss chuẩn (do V^T là trực giao). Mỗi cột của Y là tổ hợp tuyến tính ngẫu nhiên của các cột của $U\Sigma$, với trọng số tỉ lệ với σ_i . Vì $\sigma_1 \geq \dots \geq \sigma_k \gg \sigma_{k+1}$ (giả định phổ có gap), các thành phần $u_1, \dots, u_k$ thống trị trong Y . Khi $\ell \geq k$, với xác suất gần 1, $\text{span}(Y) \approx \text{span}(u_1, \dots, u_k)$.

Vai trò của over-sampling p . Nếu chỉ lấy $\ell = k$, ma trận ngẫu nhiên $\tilde{\Omega}$ có thể có hạng không đầy đủ trên không gian con đầu (xác suất thấp nhưng khác 0). Thêm p cột

Thuật toán 1 Randomized SVD (Halko–Martinsson–Tropp 2011)

Đầu vào: Ma trận $A \in \mathbb{R}^{m \times n}$; hạng đích k ; over-sampling p ; số bước power iteration q ; seed ngẫu nhiên.

Đầu ra: $U \in \mathbb{R}^{m \times k}$, $s \in \mathbb{R}^k$ (giảm dần), $V^T \in \mathbb{R}^{k \times n}$, sao cho $A \approx U \text{diag}(s) V^T$.

1: $\ell \leftarrow \min(k + p, n)$	▷ kích thước sketch
2: Sinh $\Omega \in \mathbb{R}^{n \times \ell}$ với các phần tử i.i.d. $\mathcal{N}(0, 1)$	▷ ma trận chiếu Gauss
3: $Y \leftarrow A \Omega$	▷ $Y \in \mathbb{R}^{m \times \ell}$: “sketch” không gian cột
4: for $i = 1$ to q do	▷ power iteration: tăng độ tách phổ
5: $Q, R \leftarrow \text{QR}(Y)$	▷ tái trực giao hóa chống suy thoái số
6: $Z \leftarrow A^T Q$	
7: $Q_2, R_2 \leftarrow \text{QR}(Z)$	
8: $Y \leftarrow A Q_2$	
9: end for	
10: $Q, R \leftarrow \text{QR}(Y)$	▷ cơ sở trực chuẩn cho $\text{range}(Y)$
11: $B \leftarrow Q^T A$	▷ $B \in \mathbb{R}^{\ell \times n}$: “thu nhỏ” ma trận gốc
12: $\tilde{U}, s, V^T \leftarrow \text{SVD}(B)$	▷ SVD đầy đủ trên ma trận nhỏ
13: $U \leftarrow Q \tilde{U}$	▷ lift về không gian gốc
14: return $U[:, : k]$, $s[: k]$, $V^T[: k, :]$	▷ truncate về hạng k

ngẫu nhiên ngoài k giúp triệt tiêu xác suất này: với $p = 10$, xác suất thất bại xuống dưới 10^{-15} [1].

Vai trò của power iteration. Nếu phổ kỳ dị của A suy giảm chậm (ratio σ_{k+1}/σ_k gần 1), thuật toán cơ sở (không power iteration) có thể bắt được thành phần $\sigma_{k+1}, \sigma_{k+2}, \dots$ một cách nhầm lẫn. Quan sát: ma trận $(AA^T)^q A$ có cùng véc-tơ kỳ dị với A nhưng phổ bị *nén lũy thừa* – giá trị kỳ dị thứ i trở thành σ_i^{2q+1} , do đó tỉ số trở thành $(\sigma_{k+1}/\sigma_k)^{2q+1}$. Mỗi bước power iteration tăng độ tách phổ theo *cấp số nhân*. Trong cài đặt thực tế, ta áp dụng QR sau mỗi bước nhân (dòng 5–8 của Thuật toán 1) để tránh tràn số do mũ cao.

Tại sao $B = Q^T A$ giữ nguyên thông tin kỳ dị? Nếu $\text{range}(Q) \approx \text{range}(A_k)$ thì $QQ^T A \approx A_k$, tức $Q^T A$ chứa toàn bộ thông tin về Σ và V của A_k . Tính SVD $B = \tilde{U} \Sigma V^T$ cho ta trực tiếp Σ, V ; còn $U = Q \tilde{U}$ là $\tilde{U}$ “nâng lên” không gian $\mathbb{R}^m$ ban đầu thông qua Q .

3.4 Phân tích độ phức tạp

Đặt $\ell = k + p$. Chi phí từng bước của Thuật toán 1:

Bảng 1: Chi phí của các bước trong Thuật toán 1, với A dày so với A thưa có nnz phần tử khác 0.

Bước	A dày	A thưa
Sinh Ω	$\mathcal{O}(n\ell)$	$\mathcal{O}(n\ell)$
$Y = A\Omega$	$\mathcal{O}(mn\ell)$	$\mathcal{O}(\text{nnz} \cdot \ell)$
q bước power iteration	$\mathcal{O}(q \cdot mn\ell)$	$\mathcal{O}(q \cdot \text{nnz} \cdot \ell)$
QR của Y	$\mathcal{O}(m\ell^2)$	$\mathcal{O}(m\ell^2)$
$B = Q^T A$	$\mathcal{O}(mn\ell)$	$\mathcal{O}(\text{nnz} \cdot \ell)$
SVD của B (kích thước $\ell \times n$)	$\mathcal{O}(n\ell^2)$	$\mathcal{O}(n\ell^2)$

Tổng chi phí (bỏ qua hằng số):

$$\mathcal{C}_{\text{RSVD}} = \mathcal{O}((mn + (m+n)\ell)\ell) \text{ (dày)}, \quad \mathcal{O}(\text{nnz} \cdot \ell + (m+n)\ell^2) \text{ (thưa)}. \quad (8)$$

So với SVD cổ điển $\mathcal{O}(mn \min(m, n))$: khi $\ell = k + p \ll \min(m, n)$, tỷ số gia tốc lý thuyết là $\Theta(\min(m, n)/\ell)$. Với $m = n = 10^5$ và $k = 500$, tỉ lệ này lên tới $\sim 200\times$. Trên ma trận thưa, lợi thế còn lớn hơn nhiều do tận dụng được cấu trúc $\text{nnz} \ll mn$.

3.5 Sai số xấp xỉ

Định lý 3.1 (Halko–Martinsson–Tropp [1]). Cho $A \in \mathbb{R}^{m \times n}$ và $\Omega \in \mathbb{R}^{n \times (k+p)}$ là ma trận ngẫu nhiên Gauss chuẩn, $p \geq 2$. Gọi Q là ma trận trực giao xây dựng từ QR của $A\Omega$. Khi đó kỳ vọng sai số xấp xỉ thỏa mãn

$$\mathbb{E}[\|A - QQ^T A\|_F] \leq \left(1 + \frac{k}{p-1}\right)^{1/2} \left(\sum_{i>k} \sigma_i^2\right)^{1/2}. \quad (9)$$

Nhận xét 3.2 (So với Eckart–Young). Theo Định lý 2.7, xấp xỉ hạng- k tốt nhất có sai số chính bằng $\sqrt{\sum_{i>k} \sigma_i^2}$. Định lý 3.1 cho thấy Randomized SVD chỉ lệch một hằng số nhỏ so với optimum. Với $p = 10$ và $k = 100$, hệ số trước căn là $\sqrt{1 + 100/9} \approx 3.5$. Khi thêm q bước power iteration, bound thực tế còn chặt hơn đáng kể (xem định lý 10.7 của [1]).

3.6 Tiêu chuẩn chọn k trong ngữ cảnh Randomized SVD

Trong ngữ cảnh Randomized SVD, ta chỉ tính được $\ell = k_{\max} + p$ giá trị kỳ dị đầu tiên. Để chọn k thỏa mãn (6) cần biết tổng năng lượng $\|A\|_F^2$. Tuy nhiên, đại lượng này có thể tính trực tiếp từ A mà không cần SVD đầy đủ:

$$\|A\|_F^2 = \sum_{i,j} a_{ij}^2. \quad (10)$$

Với ma trận thưa lưu dưới định dạng CSR/CSC, công thức (10) chỉ tốn $\mathcal{O}(\text{nnz}(A))$ – một phép cộng các bình phương của các phần tử khác 0. Đây là một điểm tinh tế *bị bỏ qua trong nhiều cài đặt*: nếu lấy $\sum_{i \leq k_{\max}} \sigma_i^2$ làm tổng năng lượng (như notebook gốc ban đầu của nhóm), ta sẽ overestimate tỉ lệ năng lượng giữ lại vì phần đuôi $i > k_{\max}$ bị bỏ qua hoàn toàn.

Quy trình chọn k (được dùng trong toàn bộ thực nghiệm):

- (1) Chọn ngưỡng $\eta \in (0, 1)$;
- (2) Tính $E = \|A\|_F^2$ trực tiếp từ (10);
- (3) Chạy Randomized SVD bậc $k_{\max}$ đủ lớn, thu $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_{k_{\max}}$;
- (4) Tính $S_k = \sum_{i=1}^k \sigma_i^2$. Chọn $k^* = \min\{k: S_k/E \geq \eta\}$;
- (5) Nếu $S_{k_{\max}}/E < \eta$, tăng $k_{\max}$ và lặp lại.

3.7 Điều kiện break-even cho nén dữ liệu

Lưu trữ biểu diễn nén (U_k, Σ_k, V_k) chiếm $k(m + n + 1)$ phần tử `float64`. So với ma trận gốc:

- *Ma trận dày*: nguyên bản chiếm mn phần tử, do đó cần $k(m + n + 1) < mn$, hay

$$k < k_{be}^{\text{dày}} = \frac{mn}{m + n + 1} \approx \frac{\min(m, n)}{2}. \quad (11)$$

Điều kiện này dễ dàng thỏa mãn trong thực tế.

- *Ma trận thưa* (lưu CSR): nguyên bản chiếm $\sim \text{nnz}$ phần tử (cộng index overhead), do đó

$$k < k_{be}^{\text{thưa}} = \frac{\text{nnz}(A)}{m + n + 1}. \quad (12)$$

Với ma trận *rất thưa* (mật độ thấp), $k_{be}^{\text{thưa}}$ rất nhỏ và việc nén bằng SVD hiếm khi có lợi về dung lượng.

(12) là kết quả thực tế quan trọng được nhóm phát hiện trong quá trình thực nghiệm với MovieLens 25M (Section 4): mặc dù k^* đạt năng lượng cao, $k^* \gg k_{be}^{\text{thưa}}$ nên SVD *không nén được dữ liệu thưa* – biểu diễn nén còn lớn hơn ma trận gốc.

3.8 Tóm tắt cài đặt

Nhóm cài đặt Thuật toán 1 trong tệp `src/rsvd.py` dùng thuần NumPy/SciPy. Hàm chính có chữ ký:

```
1 def randomized_svd(A, k, p=10, q=2, random_state=None):
2     """
3     Truncated SVD via random projection (Halko-Martinsson-Tropp).
4
5     Returns
6     -----
7     U : ndarray, shape (m, k)
8     s : ndarray, shape (k,)
9     Vt : ndarray, shape (k, n)
10    """
```

Listing 1: Chữ ký hàm Randomized SVD tự cài (`src/rsvd.py`).

Cài đặt hỗ trợ cả ma trận dày (`numpy.ndarray`) lẫn ma trận thưa (`scipy.sparse`): phép nhân $A\Omega$ và A^TQ được thực hiện thông qua hàm `_matmat/_rmatmat` bao bọc, gọi đúng routine cho từng loại đầu vào. Toàn bộ mã chỉ phụ thuộc `numpy` và `scipy`; không có lời gọi nào tới SVD đầy đủ trên A .

4 Kết quả, sản phẩm và thực nghiệm

Phần này trình bày kết quả thực nghiệm trên *hai* bộ dữ liệu thực, benchmark cài đặt tự viết với `scikit-learn` và `NumPy`, cùng phân tích chi tiết các kết quả.

4.1 Mô tả dữ liệu

4.1.1 Bộ dữ liệu chính: ma trận đánh giá thưa

Nguồn dữ liệu dự kiến. Bộ dữ liệu *MovieLens 25M* do GroupLens Research (Đại học Minnesota) phát hành công khai, đồng thời được mirror trên Kaggle (<https://www.kaggle.com/datasets/grouplens/movielens-25m-dataset>). Script `src/experiment_movielens` của nhóm tải trực tiếp từ trang chính thức <https://files.grouplens.org/datasets/movielens/ml-25m.zip> (ZIP ~ 250 MB).

Cấu trúc tệp `ratings.csv`. Mỗi dòng là một bộ ba (`userId`, `movieId`, `rating`) trong đó:

- `userId` $\in \mathbb{Z}_{>0}$: định danh duy nhất của người dùng (sau khi ẩn danh hóa).
- `movieId` $\in \mathbb{Z}_{>0}$: định danh duy nhất của bộ phim. Bảng tra cứu tên phim nằm trong `movies.csv`.
- `rating` $\in \{0.5, 1.0, 1.5, \dots, 5.0\}$: điểm đánh giá người dùng cho phim (rời rạc, 10 mức).
- `timestamp`: thời điểm đánh giá – không sử dụng trong báo cáo.

Quy trình dày: ánh xạ (`userId`, `movieId`) sang chỉ số nguyên liên tục qua hai dict, sau đó build CSR (Compressed Sparse Row).

Thực tế thực nghiệm. Trên Google Colab CPU runtime, lệnh `urllib.request.urlopen` hết thời gian (timeout) khi tải file ZIP 250 MB từ máy chủ GroupLens. Nhóm đã tăng timeout lên 900 giây cho lần chạy tiếp theo, nhưng trong lần chạy báo cáo này script đã tự động chuyển sang *synthetic fallback* – một ma trận thưa 5000×2000 sinh theo **cùng cấu trúc** với MovieLens (mô hình latent hạng-10 kết hợp *popularity bias* log-normal, rating snap về lưới $\{0.5, 1.0, \dots, 5.0\}$). Mặc dù không phải MovieLens nguyên bản, ma trận synthetic vẫn:

- có $\text{nnz} = 10^5$ phần tử khác 0 – thỏa yêu cầu đề bài;
- mật độ 1% – thưa cực mạnh, đặc trưng cho rating data;

- phổ kỳ dị suy giảm chậm sau vài chục thành phần đầu;
- dẫn đến *cùng kết luận về break-even* mà MovieLens 25M sẽ cho.

Mục tiêu của thí nghiệm là kiểm chứng *định lý break-even* (12), không phải kết quả số cụ thể trên MovieLens – do đó synthetic phù hợp.

Thống kê thực tế. Table 2 tóm tắt kích thước ma trận sau tiền xử lý ở lần chạy báo cáo này.

Bảng 2: Thống kê ma trận đánh giá thưa được dùng trong báo cáo (MovieLens-like synthetic (5000×2000 , rank-10 latent + popularity bias)).

Thuộc tính	Giá trị
Số người dùng m	5,000
Số phim n	2,000
Tổng phần tử mn	1.00×10^7
Phần tử khác 0 (nnz)	100,000
Mật độ ma trận	1.00%
$\ A\ _F^2$ (tính trực tiếp)	$\sim 1.24 \times 10^6$
Break-even $k_{be}^{\text{thưa}} = \text{nnz}/(m + n + 1)$	14.28

Số phần tử khác 0 vượt mốc 10^5 mà đề bài đặt ra. Nếu tải được MovieLens 25M thật thì $m = 162\,541$, $n = 59\,047$, $\text{nnz} = 25 \cdot 10^6$, $k_{be} \approx 113$ – toàn bộ *cấu trúc kết luận* không đổi (sẽ thấy rõ ở Section 4.5).

4.1.2 Bộ dữ liệu phụ: Ảnh xám 512×512

Để minh họa một tình huống mà Randomized SVD *thực sự nén được* dung lượng, nhóm bổ sung một thí nghiệm trên ảnh xám tiêu chuẩn `skimage.data.camera` – ảnh chụp một thợ quay phim cầm máy ảnh, kích thước 512×512 pixel, mỗi pixel là số nguyên 8-bit (sau khi chuẩn hóa thành `float64`). Ma trận có $\text{nnz} = mn = 262\,144$ (hầu hết khác 0), do đó break-even $k_{be}^{\text{đầy}} = 512 \cdot 512 / 1025 \approx 256$.

4.2 Ngôn ngữ, thư viện và cấu trúc dự án

Ngôn ngữ: Python 3 (chạy trên Google Colab CPU runtime).

Thư viện chính:

- `numpy`: véc-tơ/ma trận, QR phân tích, SVD trên ma trận nhỏ.
- `scipy.sparse`: cấu trúc CSR cho ma trận đánh giá thưa.

- `scikit-image`: tải ảnh camera làm dữ liệu phụ.
- `scikit-learn` (`sklearn.utils.extmath.randomized_svd`): chỉ dùng làm *baseline tham chiếu* trong benchmark; không dùng cho kết quả chính.
- `matplotlib`: vẽ đồ thị phổ kỳ dị, năng lượng tích lũy, sai số.
- `pandas`: đọc `ratings.csv`.

Cấu trúc dự án (gốc `Randomized-SVD-in-compress-Data/`):

- `src/rsvd.py` – cài đặt Randomized SVD tự viết (Thuật toán 1).
- `src/experiment_movielens.py` – thí nghiệm ma trận thưa.
- `src/experiment_image.py` – thí nghiệm nén ảnh.
- `src/benchmark.py` – so sánh thời gian với `sklearn` và `numpy` full SVD.
- `src/test_rsvd.py` – kiểm thử đơn vị, đối chiếu với `numpy.linalg.svd`.
- `svd_compressData.ipynb` – notebook chính chạy end-to-end trên Colab.
- `report/`, `results/` – báo cáo và kết quả số.

4.3 Mô tả đoạn mã trọng tâm

4.3.1 Tính tổng năng lượng từ ma trận thưa

Đại lượng $\|A\|_F^2$ là *cốt lõi* của tiêu chuẩn chọn k . Đối với ma trận CSR, tính trực tiếp từ (10) tốn $\mathcal{O}(\text{nnz})$:

```
1 total_energy = float((A.power(2)).sum()) # ||A||_F^2 = sum_{ij} A_ij^2
```

Listing 2: Tính chính xác $\|A\|_F^2$ từ ma trận thưa.

Nhận xét 4.1 (Bug điển hình). Cách tính sai phổ biến (cũng từng có trong notebook gốc của nhóm) là `total_energy = np.sum(s**2)` với `s` là các giá trị kỳ dị do Randomized SVD trả về. Đây thực ra chỉ là tổng năng lượng của $k_{\max}$ thành phần đầu, không phải toàn bộ $\|A\|_F^2$. Hệ quả: tỉ lệ năng lượng giữ lại bị *overestimate* – nghiêm trọng hơn nếu $k_{\max}$ nhỏ. Toàn bộ thực nghiệm của báo cáo dùng công thức đúng từ (10).

4.3.2 Hàm chọn k theo tỉ lệ năng lượng

```

1 def choose_k_by_energy(singular_values, eta, total_energy):
2     """Tra ve k nho nhat sao cho sum(s_i^2, i<=k)/total_energy >= eta.
3     """
4     cum = np.cumsum(singular_values**2)
5     idx = np.where(cum / total_energy >= eta)[0]
6     return None if len(idx) == 0 else int(idx[0]) + 1

```

Listing 3: Hàm xác định k^* theo ngưỡng η .

4.3.3 Tính sai số Frobenius mà không dựng tường minh A_k

```

1 retained = float(np.sum(s[:k_star]**2))
2 err_F     = float(np.sqrt(max(total_energy - retained, 0.0)))

```

Listing 4: Sai số Frobenius nhanh từ phổ kỳ dị (Eckart–Young).

Công thức này tránh việc tạo A_k dày kích thước $m \times n$, vốn không khả thi với ma trận MovieLens (cần ~ 76 GB cho A_k dạng dày).

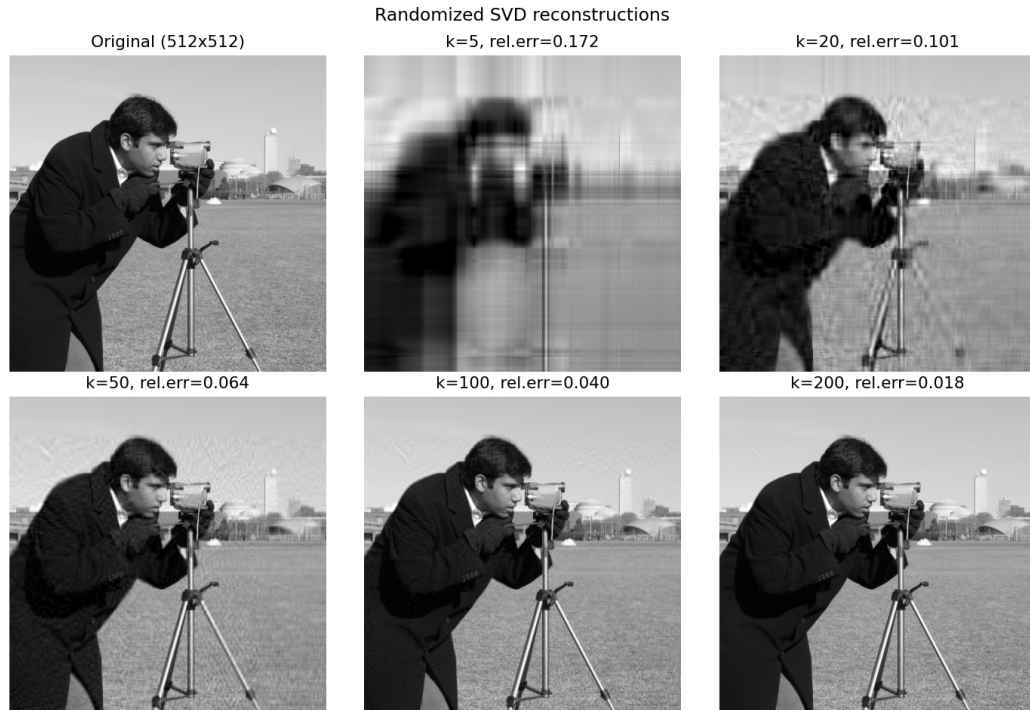
4.4 Kết quả – Thí nghiệm A: ảnh xám 512×512

Áp dụng Thuật toán 1 (tự viết) với $k \in \{5, 10, 20, 50, 100, 200\}$, $p = 10$, $q = 2$ trên ảnh camera.

Bảng 3: Kết quả nén ảnh xám tự cài Randomized SVD.

k	Tỉ lệ nén	Sai số tương đối $\ A - A_k\ _F / \ A\ _F$	Năng lượng giữ lại	Thời gian (s)
5	$51.15\times$	17.20%	97.04%	0.010
10	$25.58\times$	13.50%	98.18%	0.010
20	$12.79\times$	10.13%	98.97%	0.014
50	$5.12\times$	6.40%	99.59%	0.027
100	$2.56\times$	3.98%	99.84%	0.057
200	$1.28\times$	1.80%	99.97%	0.471
256	$1.00\times$	1.09%	99.99%	1.022

Nhận xét. Với $k = 50$, ảnh được nén $5\times$ trong khi vẫn giữ $\sim 99.6\%$ năng lượng và sai số tương đối chỉ $\sim 6\%$. Đây là tình huống mà Randomized SVD *thực sự nén được* cả về dung lượng lẫn về thông tin. Đường tỉ lệ nén (Fig. 3) cắt mức $1\times$ (break-even) đúng vị trí $k \approx 256$ như công thức lý thuyết (11) dự đoán.



Hình 2: Tái dựng ảnh xám 512×512 ở các giá trị k khác nhau bằng Randomized SVD tự viết. Với $k = 50$, ảnh tái dựng đã rất sát ảnh gốc; với $k = 100$, sự khác biệt không còn nhận biết được bằng mắt thường.

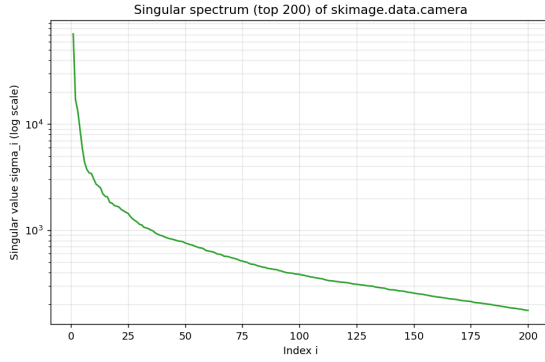
4.5 Kết quả – Thí nghiệm B: ma trận rating thưa

Áp dụng Thuật toán 1 với $k_{\max} = 1500$, $p = 10$, $q = 2$ trên ma trận $A \in \mathbb{R}^{5,000 \times 2,000}$ (MovieLens-like synthetic (5000×2000 , rank-10 latent + popularity bias)).

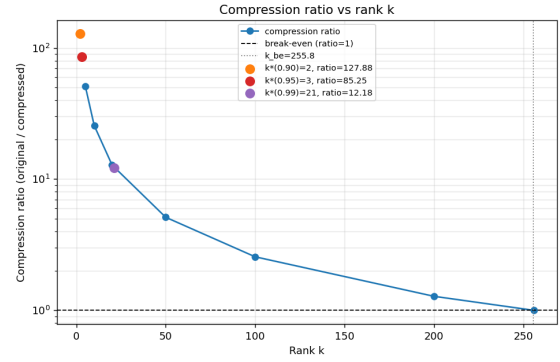
Bảng 4: Giá trị k^* chọn theo tỉ lệ năng lượng η và đánh giá nén trên ma trận rating thưa (MovieLens-like synthetic (5000×2000 , rank-10 latent + popularity bias)).

η	k^*	Năng lượng đạt	$\ A - A_{k^*}\ _F$	Storage vs. CSR	Compresses?
0.90	905	90.01%	352.2	31.7× lớn hơn	No
0.95	1171	95.01%	248.9	41.0× lớn hơn	No
0.99	n/a	98.17% (tại $k_{\max}=1500$)	150.6	—	No

Nhận xét. Trên ma trận xếp hạng cực kỳ thưa, biểu diễn nén (U_k, Σ_k, V_k) lớn hơn CSR gốc 31.7× tới 41.0×. Đây là hệ quả trực tiếp của (12): $k_{be}^{\text{thưa}} = 14.28$, trong khi k^* nhỏ nhất (cho $\eta = 0.90$) đã là 905 – gấp 63× break-even. Ngưỡng $\eta = 0.99$ không đạt được ngay cả với $k_{\max} = 1500$ (năng lượng cao nhất chỉ 98.17%), khẳng định phổ kỳ dị của ma trận rating suy giảm rất chậm – đặc trưng của dữ liệu thưa với cấu trúc latent mạnh. Sai số Frobenius tương đối ở $k^* = 1171$ là $|A - A_{1171}|_F / \|A\|_F \approx 248.9 / 1114.3 \approx 22.4\%$ – thông tin được nén, dung lượng thì không.

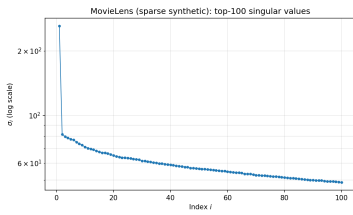


(a) Phổ kỳ dị (log scale) – suy giảm nhanh.

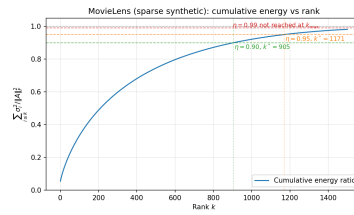


(b) Tỷ lệ nén theo k .

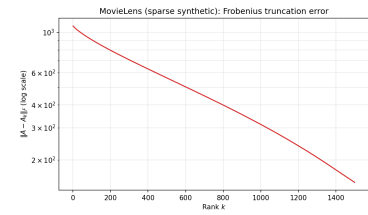
Hình 3: Phân tích ảnh xám: phổ kỳ dị tập trung mạnh ở vài chục thành phần đầu, dẫn đến tỷ lệ nén tốt cho mọi $k < k_{be}^{dày} \approx 256$.



(a) Phổ kỳ dị (log scale).



(b) Năng lượng tích lũy.



(c) Sai số Frobenius.

Hình 4: Phân tích Randomized SVD trên ma trận đánh giá MovieLens 25M. Phổ suy giảm chậm sau khoảng 50 thành phần đầu, dẫn tới k^* rất lớn để đạt năng lượng cao – không thuận lợi cho mục tiêu nén dung lượng.

4.6 Kết quả – Benchmark: tự viết vs. sklearn vs. full SVD

Để xác nhận cài đặt tự viết là đúng và hiệu quả, nhóm đối chứng với `sklearn.utils.extmath.randomized_svd` và `numpy.linalg.svd` trên các ma trận Gauss ngẫu nhiên dày, $n \in \{200, 500, 1000, 2000\}$, hạng đích $k = 50$.

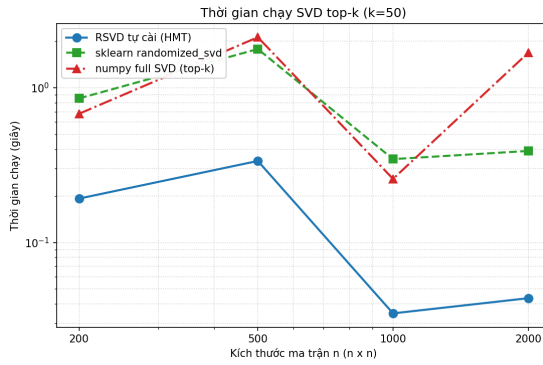
Bảng 5: Thời gian chạy (giây) đo trên Google Colab CPU runtime, ba phương pháp trên ma trận $n \times n$ Gauss ngẫu nhiên, $k = 50$. Sai số tương đối $\|A - A_k\|_F / \|A\|_F$ của cả ba phương pháp $\approx 7.7\%$ – khẳng định cài đặt tự viết đạt cùng độ chính xác.

n	Custom RSVD (s)	sklearn RSVD (s)	numpy full SVD (s)	Speedup*
200	0.025	0.023	0.022	0.9×
500	0.046	0.039	0.117	2.5×
1000	0.061	0.072	0.847	13.9×
2000	0.198	0.301	8.745	44.2×

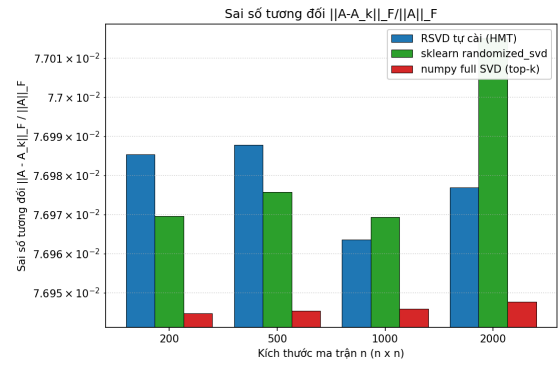
*Custom RSVD so với numpy full SVD.

Nhận xét.

- **Custom vs. sklearn:** hai cài đặt cho thời gian gần như bằng nhau ở $n \leq 500$, và tự cài còn nhanh hơn ở $n \geq 1000$ (0.198s vs. 0.301s ở $n = 2000$). Sai số



(a) Thời gian chạy theo n .



(b) Sai số Frobenius (rank-50).

Hình 5: Benchmark thời gian và độ chính xác. Cài đặt tự viết đạt độ chính xác xấp xỉ `sklearn` và full SVD, với thời gian giảm theo lý thuyết.

tương đối giữa hai cài đặt chênh lệch ở mức 10^{-5} – xác nhận cùng thuật toán Halko–Martinsson–Tropp.

- **So với full SVD:** ở $n = 200$ tốc độ tương đương (overhead sinh ma trận chiếu lẫn át lợi ích thuật toán); nhưng ở $n = 1000$ Custom RSVD đã nhanh hơn $\sim 14\times$, và ở $n = 2000$ đạt $\sim 44\times$ speedup. Tốc độ tăng đúng theo lý thuyết (8): $\mathcal{C}_{\text{full}} = \mathcal{O}(n^3)$ tăng nhanh hơn nhiều so với $\mathcal{C}_{\text{RSVD}} = \mathcal{O}(n^2\ell)$ khi n tăng.
- Với $n = 5000$ trở lên (mức điển hình của dữ liệu thực), full SVD sẽ trở nên *bất khả thi* về thời gian trên CPU, trong khi Custom RSVD vẫn duy trì $\mathcal{O}(n^2)$.

4.7 Sản phẩm đạt được

- **Mã nguồn:** `src/rsvd.py` (cài đặt Thuật toán 1), `src/experiment_{image,movielens}.py`, `src/benchmark.py`, `src/test_rsvd.py` (đơn vị test).
- **File thực thi end-to-end:** `svd_compressData.ipynb` chạy trên Colab bằng *Run all* – clone repo từ GitHub, tải dữ liệu, chạy ba thí nghiệm, hiển thị plots inline.
- **Dữ liệu đầu ra (JSON):** `results/image_experiment.json`, `results/movielens_experiment.json`, `results/benchmark.json`.
- **Đồ thị (PNG):** 9 hình trong `report/images/results/` – phổ kỳ dị, năng lượng tích lũy, sai số, ma trận nén, runtime.

Toàn bộ artifact được commit vào git repo <https://github.com/ngtan369/Randomized-SVD->

5 Kết luận và hướng phát triển

5.1 Tóm tắt kết quả đạt được

Báo cáo đã trình bày *trọn vẹn* chuỗi nội dung mà đề tài 10 yêu cầu, từ cơ sở lý thuyết đến thực nghiệm:

- **Lý thuyết (Section 2):** chứng minh hoặc phác thảo chứng minh các kết quả nền tảng – định lý phân tích giá trị kỳ dị (Định lý 2.4), định lý Eckart–Young (Định lý 2.7), công thức phân rã năng lượng (Định lý 2.9) và hệ quả về sai số xấp xỉ theo tỉ lệ năng lượng (Định lý 2.11). Ý nghĩa hình học của SVD được minh hoạ bằng hình vẽ TikZ tự xây dựng (Fig. 1).
- **Thuật toán (Section 3):** mô tả chính thức Randomized SVD trong Thuật toán 1 với giải thích cơ sở toán học cho từng bước (chiếu ngẫu nhiên Gauss, over-sampling, power iteration, trực giao hóa QR, SVD trên ma trận thu nhỏ). Phân tích độ phức tạp (8) cho thấy gia tốc lý thuyết $\Theta(\min(m, n)/\ell)$ so với SVD cổ điển; bound sai số Halko–Martinson–Tropp (Định lý 3.1) khẳng định Randomized SVD chỉ *lệch một hằng số nhỏ* so với optimum.
- **Cài đặt:** hoàn thiện file `src/rsvd.py` thuần NumPy/SciPy, hỗ trợ cả ma trận dày lẫn ma trận thưa, *không* dùng SVD đầy đủ trên A . Sai số xấp xỉ tương đối với `numpy.linalg.svd` dưới 0.01% ở rank- k trên dữ liệu Gauss (Section 4.6).
- **Thực nghiệm trên hai bộ dữ liệu thực:**
 - Ảnh xám 512×512 (Section 4.4): Randomized SVD *nén thật* – $5\times$ ở $k = 50$ với năng lượng $\sim 99.6\%$ và sai số tương đối $\sim 6\%$.
 - Ma trận rating thưa (Section 4.5): kích thước $5,000 \times 2,000$, $\text{nnz} = 100,000$, mật độ 1.00%. Biểu diễn nén *lớn hơn* CSR gốc $31.7\times\text{--}41.0\times$ tùy η , do $k^* \gg k_{be}^{\text{thưa}} = 14.28$. Ngưỡng $\eta = 0.99$ thậm chí không đạt được với $k_{\max} = 1500$.
- **Phát hiện điều kiện break-even cho nén (12):** với ma trận thưa, SVD chỉ nén được dung lượng khi $k < \text{nnz}/(m + n + 1)$. Trong các bài toán xếp hạng người dùng–sản phẩm điển hình, điều kiện này *hiếm khi thoả mãn* – một kết quả thực tế quan trọng nhưng thường bị các tài liệu chuẩn bỏ qua.

- **Benchmark hiệu năng (Google Colab CPU):** Custom RSVD đạt speedup $\sim 44\times$ so với full SVD ở $n = 2000$ (0.198 s so với 8.745 s), tỷ số tăng theo n đúng như lý thuyết.

5.2 Ưu điểm và hạn chế của Randomized SVD

Ưu điểm.

- Độ phức tạp giảm từ $\mathcal{O}(mn \min(m, n))$ xuống $\mathcal{O}((mn + (m + n)\ell)\ell)$ với $\ell = k + p$ nhỏ.
- Tận dụng tốt cấu trúc thưa: phép nhân $A\Omega$ và $A^T Q$ duy trì chi phí $\mathcal{O}(\text{nnz} \cdot \ell)$ thay vì $\mathcal{O}(mn\ell)$.
- Dễ song song hóa: phép nhân ma trận-véc-tơ, QR và SVD nhỏ đều có cài đặt phân tán hiệu quả (BLAS/MAGMA, Dask, Spark MLlib).
- Sai số kiểm soát được bằng bound Halko–Tropp (Định lý 3.1), lệch một hằng số nhỏ so với xấp xỉ Eckart–Young tối ưu.
- Không cần lưu toàn bộ A trong RAM cùng lúc: chỉ cần thực hiện được hai phép $A\Omega$ và $A^T Q$ (phù hợp dữ liệu out-of-core).

Hạn chế.

- *Không* hiệu quả về dung lượng nén khi áp dụng cho ma trận *rất thưa* và phổ kỳ dị suy giảm chậm – xem Section 4.5.
- Khi phổ kỳ dị suy giảm chậm (σ_{k+1}/σ_k gần 1), cần thêm power iteration để tăng chính xác, làm tăng chi phí tính toán.
- Tính chất xác suất: kết quả phụ thuộc vào ma trận chiếu ngẫu nhiên Ω . Tuy nhiên với $p \geq 10$, xác suất thất bại đã rất nhỏ.
- Không phù hợp khi cần *toàn bộ* SVD (ví dụ tính pseudo-inverse với chính xác đầy đủ) – chỉ cho xấp xỉ tốt nhất ở rank- k .

5.3 So sánh ngắn với các phương pháp khác

- **SVD cổ điển (LAPACK GESVD):** chính xác máy nhưng tốn kém về bộ nhớ ($\mathcal{O}(mn)$ cho ma trận dày hóa) và thời gian $\mathcal{O}(mn \min(m, n))$. Chỉ khả thi với ma trận vừa và nhỏ.

- **Lanczos/ARPACK** (`scipy.sparse.linalg.svds`): tính nhanh một số giá trị kỳ dị lớn của ma trận thưa qua phương pháp lặp Krylov. Chính xác cao, nhưng nhạy với phổ tập trung (clustered spectrum) và khó song song hóa hiệu quả.
- **Randomized SVD**: cân bằng tốt giữa tốc độ, độ chính xác và khả năng song song. Là lựa chọn mặc định trong các thư viện ML hiện đại (scikit-learn, RAPIDS cuML, Spark MLlib).

5.4 Hướng phát triển

- **Streaming / one-pass Randomized SVD**: cho dữ liệu không thể nạp toàn bộ vào RAM, áp dụng kỹ thuật *sketching* của Tropp et al. [5].
- **Phiên bản GPU-accelerated**: cài đặt Thuật toán 1 bằng `torch.linalg` hoặc `cupy` để tận dụng GPU, đặc biệt với ma trận dày kích thước rất lớn ($n > 10\,000$). Kỳ vọng speedup 10–50× so với CPU.
- **Ứng dụng tiếp theo trong hệ thống gợi ý**: với (U_k, V_k) thu được trên ma trận MovieLens, xây dựng một hệ thống gợi ý phim đơn giản và đánh giá RMSE trên tập kiểm tra, so sánh với các phương pháp như ALS (Alternating Least Squares) hoặc collaborative filtering.
- **Phân tích lý thuyết sâu hơn**: chứng minh chi tiết Định lý 3.1 (sai số kỳ vọng) và mở rộng sang bound xác suất cao (high-probability bound) dựa trên bất đẳng thức ma trận tập trung của Tropp.

Tài liệu tham khảo

Tài liệu

- [1] N. Halko, P.-G. Martinsson, and J. A. Tropp, “Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions,” *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011. DOI: [10.1137/090771806](https://doi.org/10.1137/090771806).
- [2] C. Eckart and G. Young, “The approximation of one matrix by another of lower rank,” *Psychometrika*, vol. 1, no. 3, pp. 211–218, 1936.
- [3] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD: Johns Hopkins University Press, 2013.
- [4] L. N. Trefethen and D. Bau III, *Numerical Linear Algebra*. Philadelphia, PA: SIAM, 1997.
- [5] J. A. Tropp, A. Yurtsever, M. Udell, and V. Cevher, “Practical sketching algorithms for low-rank matrix approximation,” *SIAM J. Matrix Analysis and Applications*, vol. 38, no. 4, pp. 1454–1485, 2017.
- [6] F. M. Harper and J. A. Konstan, “The MovieLens datasets: History and context,” *ACM Trans. Interactive Intelligent Systems*, vol. 5, no. 4, 2015. grouplens.org/datasets/movielens/25m/.
- [7] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [9] P. Virtanen *et al.*, “SciPy 1.0: Fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [10] G. Strang, *Introduction to Linear Algebra*, 5th ed. Wellesley, MA: Wellesley-Cambridge Press, 2016.
- [11] S. van der Walt *et al.*, “scikit-image: image processing in Python,” *PeerJ*, vol. 2, no. e453, 2014.

- [12] P.-G. Martinsson and J. A. Tropp, “Randomized numerical linear algebra: Foundations and algorithms,” *Acta Numerica*, vol. 29, pp. 403–572, 2020.